

PHYS 771 Assignment 1 due June 17 2026 11:59pm

The goal of this exercise will be to calculate the radial velocity curve of a planetary system discovered by the Kepler satellite (Crossfield et al. 2015; <http://arxiv.org/abs/1501.03798>). You will study the feasibility of measuring the masses of these planets with a high-resolution spectrometer that reaches the meter per second precision. These three Super-Earth planets were discovered around an M-type low-mass star (named K2-3) with the transit method. The physical parameters of the planets are given in Table 2 of the paper. For this exercise, we will suppose that all three planets have co-planar and circular orbits, with an inclination of exactly 90 degrees. We will also suppose that the impact parameter of the transit is null and that all arguments of pericentre are equal at 90 degrees.

```
In [1]: import astropy.units as u
import astropy.constants as const
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from scipy.optimize import curve_fit
from tqdm import tqdm
```

a) Determine the masses of the three planets, in units of Earth masses. Assume that K2-3 b and K2-3 c have volumetric densities equal to that of Jupiter (1.33 g/cm³) and that K2-3 d has a volumetric density equal to that of the Earth (5.5 g/cm³).

I define the given values of volumetric densities (from the question) and radii (from the paper). I define the functions used to obtain planet masses from these values. I chose to use the mass-radius relationships from the paper.

```
In [2]: # volumetric densities assumptions
rho_K2b = 1.33 * u.g / u.cm**3
rho_K2c = 1.33 * u.g / u.cm**3
rho_K2d = 5.5 * u.g / u.cm**3
```

```
In [3]: # planet radii from Table 2 of the paper
R_K2b = 2.14 * u.earthRad
R_K2c = 1.72 * u.earthRad
R_K2d = 1.52 * u.earthRad
```

```

In [4]: # Mass-radius relations from section 2.6 of the paper
def MassRadius(radius, density):
    """
    radius: Rearth
    density: g/cm**3
    """
    if radius < 1.5 * u.earthRad:
        mass = ((4*np.pi)/3 ) * radius**3 * density

        if (mass.cgs.unit != u.g):
            print("Units error")
            return None

        return mass.to(u.earthMass)

    if radius >= 1.5 * u.earthRad and radius < 4.0 * u.earthRad:
        mass = (2.69 * u.earthMass) * (radius.value)**0.93

        if (mass.cgs.unit != u.g):
            print("Units error")
            return None

        return mass.to(u.earthMass)

    if radius >= 4.0 * u.earthRad:
        mass = (1.0 * u.earthMass) * (radius.value)**2.06

        if (mass.cgs.unit != u.g):
            print("Units error")
            return None

        return mass.to(u.earthMass)

    print('Mistakes have been made.')
    return None

```

```

In [5]: # We are given densities and I realize that since the radii of all thr
# then we end up not using the density and that tells me that maybe we

# I included this function for testing but decided to stick with my in
def sphericMassRadius(radius, density):
    mass = ((4*np.pi)/3 ) * radius**3 * density

    if (mass.cgs.unit != u.g):
        print("Units error")
        return None

    return mass.to(u.earthMass)

```

```

In [6]: # compute masses from the relations above
M_K2b = MassRadius(R_K2b, rho_K2b)

```

```

M_K2c = MassRadius(R_K2c, rho_K2c)
M_K2d = MassRadius(R_K2d, rho_K2d)

# Assuming constant density spheres, unused btu included for testing a
#M_K2b = sphericMassRadius(R_K2b, rho_K2b)
#M_K2c = sphericMassRadius(R_K2c, rho_K2c)
#M_K2d = sphericMassRadius(R_K2d, rho_K2d)

```

```

In [7]: print(f"Mass of K2-3b: {M_K2b:.3f}")
        print(f"Mass of K2-3c: {M_K2c:.3f}")
        print(f"Mass of K2-3d: {M_K2d:.3f}")

```

Mass of K2-3b: 5.458 earthMass

Mass of K2-3c: 4.454 earthMass

Mass of K2-3d: 3.971 earthMass

b) Determine the expected radial velocity semi-amplitude (K) for each planet.

I define my constants: Periods and star mass taken from the paper; the inclinations and eccentricities are given assumptions from the assignment

```

In [8]: # star mass from Table 1
        Mstar = 0.601 * u.solMass

        # inclination and eccentricities are given assumptions
        i = 90.0
        e = 0.0

        # planet periods from Table 2
        P_K2b = 10.05403 * u.day
        P_K2c = 24.6454 * u.day
        P_K2d = 44.5631 * u.day

```

I use the Radial Velocity formula given in the lecture notes to compute the semi-amplitude of each planet

```

In [9]: # We use the formula from the RV lecture notes (2.26)
        def semiAmplitude(period, Mp, Mstar, i, e):
            """
            Period: days
            Mp: earthMass
            Mstar: solMass
            i: unitless float
            e: unitless float
            """

            K = ((2*np.pi*const.G)/period)**(1/3) * (Mp * np.sin(np.deg2rad(i)

            if (K.cgs.unit != (u.cm/u.s)):

```

```

        print("Units error")
        return None

    return K.to(u.m/u.s)

```

```

In [10]: # compute RV semi-amplitudes using the formula
K_K2b = semiAmplitude(P_K2b, M_K2b, Mstar, i, e)
K_K2c = semiAmplitude(P_K2c, M_K2c, Mstar, i, e)
K_K2d = semiAmplitude(P_K2d, M_K2d, Mstar, i, e)

```

```

In [11]: print(f"RV Semi-amplitude of K2-3b: {K_K2b:.3f}")
        print(f"RV Semi-amplitude of K2-3c: {K_K2c:.3f}")
        print(f"RV Semi-amplitude of K2-3d: {K_K2d:.3f}")

```

RV Semi-amplitude of K2-3b: 2.271 m / s

RV Semi-amplitude of K2-3c: 1.374 m / s

RV Semi-amplitude of K2-3d: 1.006 m / s

c) Determine the radial velocity of the star K2-3 with respect to an observer on Earth, in meters per second at moment $t = T_{0c}$, where T_{0c} is the epoch of central transit of the planet K2-3 c (given in Table 2 of the paper). You can ignore the radial velocity of the center of mass of the K2-3 system, and the Earth's motion.

I define my epoch of central transit as taken from the paper, and define the given assumptions about systemic velocity and argument of pericenter

```

In [12]: # epoch of central transit of each planet from Table 2
T_0b = 1980.4189 * u.day
T_0c = 1979.2786 * u.day
T_0d = 1993.2232 * u.day

# ignored COM motion and Earth motion
v_sys = 0.0 * (u.m / u.s)

# given that the argument of pericenter is 90 deg
w = 90.0

```

For a circular orbit ($e = 0$), the true anomaly (ν) is equal to the mean anomaly (M), which simplifies calculations by taking $\nu = M$

I compute the true anomaly for each planet using the mean anomaly formula

```

In [13]: # since we are assuming circular orbits, the true anomaly = mean anomaly
def meanAnomaly(period, t, t_0):
    """
    period: days
    t: days (time we are evaluating at)

```

```

t_0: days (transit time OF THE PLANET)
"""
nu = (2*np.pi/period) * (t - t_0)

if (nu.cgs.unit != u.dimensionless_unscaled):
    print("Units error")
    return None
return nu * u.rad

```

```

In [14]: # compute the anomalies of each planet at planet c's transit time
nu_K2b_T0c = meanAnomaly(P_K2b, T_0c, T_0b)
nu_K2c_T0c = meanAnomaly(P_K2c, T_0c, T_0c)
nu_K2d_T0c = meanAnomaly(P_K2d, T_0c, T_0d)

```

```

In [15]: print(f"True anomaly of K2-3b at T_0c: {nu_K2b_T0c:.3f}")
print(f"True anomaly of K2-3c at T_0c: {nu_K2c_T0c:.3f}")
print(f"True anomaly of K2-3d at T_0c: {nu_K2d_T0c:.3f}")

```

```

True anomaly of K2-3b at T_0c: -0.713 rad
True anomaly of K2-3c at T_0c: 0.000 rad
True anomaly of K2-3d at T_0c: -1.966 rad

```

The star's RV will be the sum of the contribution of all three planets, so I compute the RV from each planet independently and sum them afterwards

The equation for the star's RV caused by a planet takes into account eccentricity and systemic velocity but we are told that both of these are zero which simplifies the calculations

I write this function to take in $\nu(t)$ evaluated at a given time and define it for a single planet

```

In [16]: # RV of the star due to a planet is given by eq 2.29 from the RV slide
# N.B. I ignored the long-term linear drift term since we are told to
def starRVfromPlanetatTime(K, w, nu, v_sys, e):
    """
    K: RV semi-amplitude (m/s)
    w: rad (arg of pericenter)
    nu: rad (anomaly EVALUATED AT TIME t)
    v_sys: m/s (systemic velocity)
    e: unitless (eccentricity)
    """

    RV = K * ( np.cos(np.deg2rad(w) + nu.value) + e*np.cos(np.deg2rad(

    if (RV.cgs.unit != (u.cm/u.s)):
        print("Units error")
        return None

    return RV.to(u.m/u.s)

```

```
In [17]: # Compute the starRV due to each planet
starRV_K2b_T0c = starRVfromPlanetatTime(K_K2b, w, nu_K2b_T0c, v_sys, e
starRV_K2c_T0c = starRVfromPlanetatTime(K_K2c, w, nu_K2c_T0c, v_sys, e
starRV_K2d_T0c = starRVfromPlanetatTime(K_K2d, w, nu_K2d_T0c, v_sys, e
```

```
In [18]: print(f"RV of star due to K2-3b at T_0c: {starRV_K2b_T0c:.3f}")
print(f"RV of star due to K2-3c at T_0c: {starRV_K2c_T0c:.3f}")
print(f"RV of star due to K2-3d at T_0c: {starRV_K2d_T0c:.3f}")
```

RV of star due to K2-3b at T_0c: 1.485 m / s
 RV of star due to K2-3c at T_0c: 0.000 m / s
 RV of star due to K2-3d at T_0c: 0.928 m / s

```
In [19]: # Add all the contributions from each planet to obtain the total star
starRV_total_T0c = starRV_K2b_T0c + starRV_K2c_T0c + starRV_K2d_T0c

print(f"Total RV of star due to K2-3b,c,d at T_0c: {starRV_total_T0c:.3f}")
```

Total RV of star due to K2-3b,c,d at T_0c: 2.413 m / s

d) Plot the radial velocity curve of K2-3 as a function of time in a period of 150 days, starting at T0c. What is the peak-to-peak amplitude of the radial velocity curve ? List your radial velocity values at $t = T_0c + \Delta t$ with $\Delta t = 0, 2, 4, 8, 16, 32, 64$ and 128 days.

I set up my observation times (the x-axis of my plots) as well as my diagnostic times

```
In [20]: # time values to print out the RV at
print_out_times = u.Quantity(T_0c.value + np.array([0,2,4,8,16,32,64,128], dtype=int), u.day)

# time array for x axis
t_array = u.Quantity(T_0c.value + np.arange(0,151,1,dtype=int), u.day)
```

I define arrays for the Periods, Transit Timings, and Semi-amplitudes of all three planets. This makes it easier to loop over all three when computing values

```
In [21]: # array of planet periods
P_array = u.Quantity([P_K2b, P_K2c, P_K2d], u.day)

# array of planet transit times
t_0_array = u.Quantity([T_0b, T_0c, T_0d], u.day)

# array of planet RV semi-amplitudes
K_array = u.Quantity([K_K2b, K_K2c, K_K2d], u.m/u.s)
```

I redefine my star RV function to loop over all the planets' contributions and to sum them in one function call to simplify my calculations. I could've done this

from the start but this will do

```
In [22]: # Total star RV as a function of time function
# N.B. I should've just made it this way in part c but that's okay
def TotalStarRV(K_array, w, v_sys, e, periods_array, t, t_0_array):
    """
    K_array: RV semi-amplitude (m/s)
    w: rad (arg of pericenter)
    v_sys: m/s (systemic velocity)
    e: unitless (eccentricity)

    periods_array: array of planet's period in days
    t: days
    t_0_array: array of transit times in days
    """
    RVtot = 0.0 * (u.m/u.s)

    for i in range(len(periods_array)):
        RV_planet = K_array[i] * ( np.cos(np.deg2rad(w) + meanAnomaly(

        RVtot = RVtot + RV_planet

    RVtot = RVtot + v_sys # correct for v_sys only once (offsets the e

    if (RVtot.cgs.unit != (u.cm/u.s)):
        print("Units error")
        return None

    return RVtot.to(u.m/u.s)
```

Evaluate the star RV at all the given observation times

```
In [23]: # Evaluate total star RV over 150 days
starRVtot = TotalStarRV(K_array, w, v_sys, e, P_array, t_array, t_0_ar
```

```
In [24]: # Plot the star total RV as a function of time
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

ax.axhline(0, color='gray', lw=1, ls="--", alpha=0.3)
ax.plot(t_array, starRVtot, color="black", lw=2, label="Total model RV

ax.set_xlabel("Time starting from K2-3c central transit [days]", fonts
ax.set_ylabel("Radial velocity [m/s]", fontsize=12)
ax.set_title("K2-3 Radial Velocity Curve", fontsize=14, fontweight="bo

ax.set_xlim(t_array[0].value, t_array[-1].value)

# make y-limits symmetric around zero
rv_absmax = max(abs(starRVtot.value.min()), abs(starRVtot.value.max()))
ax.set_ylim(-1.15 * rv_absmax, 1.15 * rv_absmax)
```

```

# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, w
ax.tick_params(axis="both", which="minor", direction="in", length=3, w
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)

ax.legend(frameon=False, fontsize=11, loc="best")

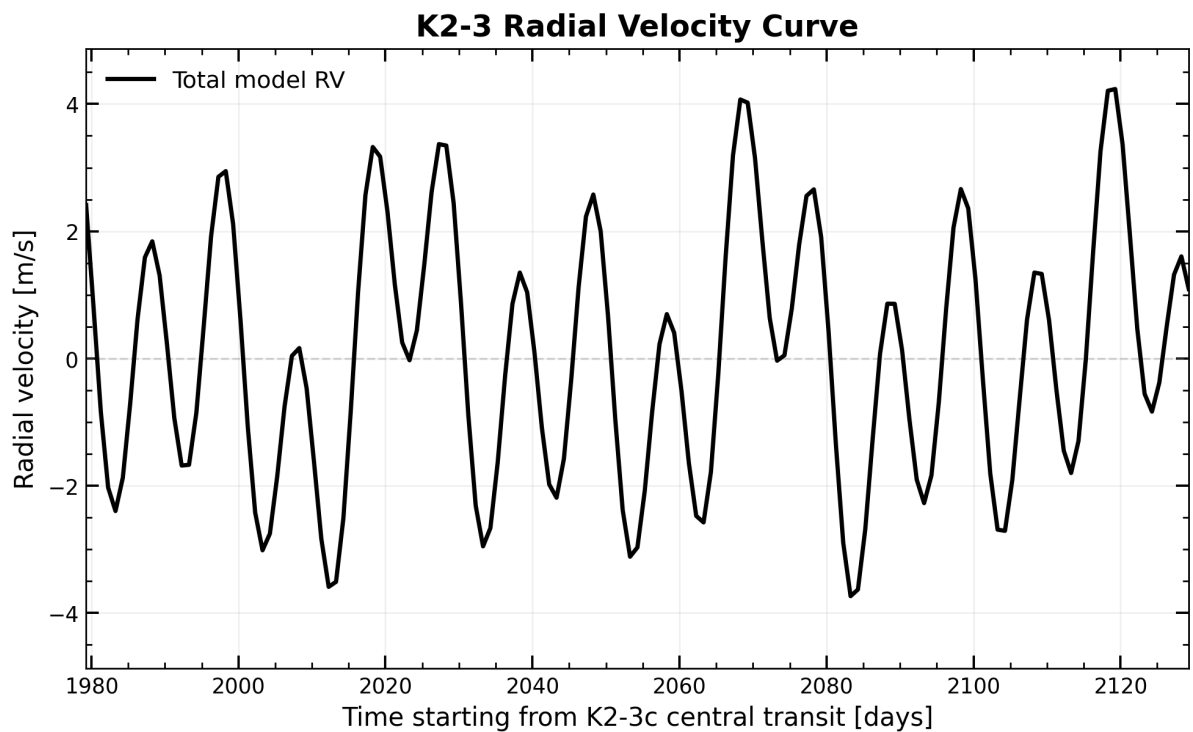
fig.tight_layout()
plt.show()

# Print asked values
print("\nTotal RV of K2-3 due to K2-3b, c, and d")
print("-" * 40)
print(f"{'Time [days]':>12} | {'RV [m/s]':>13}")
print("-" * 40)

for time in print_out_times:
    print(f"{time.value:12.3f} | {starRVtot[np.where(t_array == time)]")

print("-" * 40)

```



Total RV of K2-3 due to K2-3b, c, and d

Time [days]	RV [m/s]
1979.279	2.413
1981.279	-0.834
1983.279	-2.397
1987.279	1.590
1995.279	0.507
2011.279	-2.835
2043.279	-2.188
2107.279	0.609

e) Simulate some observations of this radial velocity curve on the span of the 150days used in d). The sampling of the observations is one measurement per day at a precision of 1 m/s. To simulate the measurement errors, draw random noise from a Gaussian distribution centered on the true radial velocity at each epoch, with a standard deviation of 1 m/s. Plot these simulated measurements (with the appropriate error bars) on top of the expected radial velocity curve.

I define my noise as gaussian noise centered at zero (so it adds directly on top of my measurements) and with a standard deviation of 1 m/s so that it simulates my 1 m/s instrumental precision. Afterwards I plot the noisy data on the same plot as the ideal model RV curve I computed earlier (N.B. this is not a fit to the data)

```
In [25]: # Gaussian parameters
stdev = 1.0 #m/s
mean = 0.0 #will center on RV value when added on top of data
sampling = np.arange(0,151,1,dtype=int) # 1 measurement per day for 150 days

# Gaussian noise curve
rng = np.random.default_rng(0) # fixed seed so the plot is reproducible
# set seed just makes it reproducible

noise = rng.normal(
    loc=mean,
    scale=stdev,
    size=len(sampling)
) * (u.m / u.s)

# Data with noise added
starRV_obs = starRVtot + noise
```

```
In [26]: # Error bars setup
RV_err = np.ones_like(t_array.value) * (stdev * u.m / u.s)
```

```

In [27]: # Plot the star total RV as a function of time with error bars
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

ax.plot(t_array, starRVtot, color="red", lw=2, label="Total Model RV",
# Error bars
ax.errorbar(
    t_array.value,
    starRV_obs.value,
    yerr=RV_err.value,
    fmt="o",
    ms=3.0,
    color='black',
    alpha=.8,
    capsize=2,
    label="Total Observed Noisy RV",
    #lw = 1
)
ax.axhline(0, color='gray', lw=1, ls="--", alpha=0.3)

ax.set_xlabel("Time starting from K2-3c central transit [days]", fontsize=12)
ax.set_ylabel("Radial velocity [m/s]", fontsize=12)
ax.set_title("K2-3 Radial Velocity Curve with Gaussian Noise", fontsize=12)

ax.set_xlim(t_array[0].value, t_array[-1].value)

# make y-limits symmetric around zero
rv_absmax = max(abs(starRV_obs.value.min()), abs(starRV_obs.value.max()))
ax.set_ylim(-1.25 * rv_absmax, 1.25 * rv_absmax)

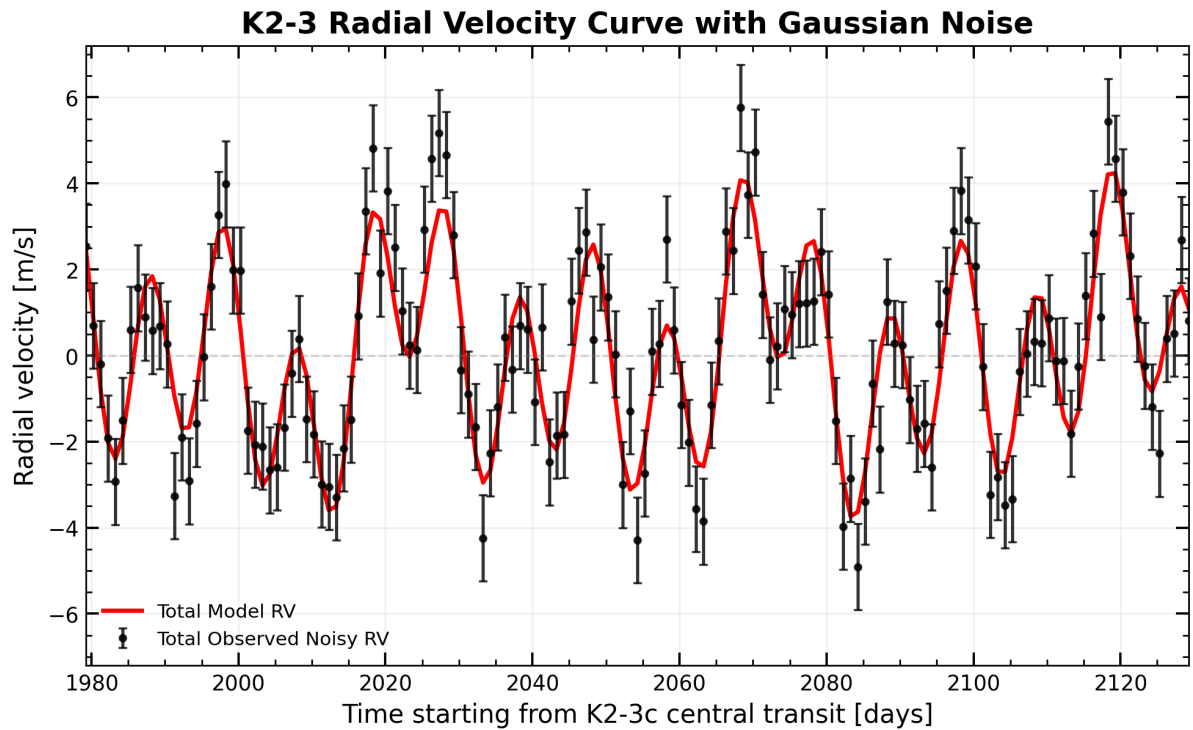
# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, width=1)
ax.tick_params(axis="both", which="minor", direction="in", length=3, width=1)
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)

ax.legend(frameon=False, fontsize=9, loc="best")

fig.tight_layout()
plt.show()

```



f) What periodogram algorithm would be most appropriate for this problem, and why? Use any software or code of your choice to calculate the periodogram of your simulated radial velocity curve, and see if you can detect the periodic signal associated with each planet.

After some quick research I've found a couple of options: Fourier Transforms (FFT), Welch Method, Lomb-Scargle Periodogram (LSP), Box-fitting Least squares (BLS), etc.

FFT is best adapted to evenly spaced data which eliminates it. Welch's method needs a continuous signal which I don't think works well in astrophysics where we can't get continuous data most of the time, also this case has sampling too low to be considered continuous in my opinion. It seems to me like LSP and BLS are the most well adapted for astrophysical time-series data that is unevenly spaced and with varying resolution. BLS fits to a box shape which makes it well suited for transits but not quite right for this case. LSP is the best option as it evaluates data points using a least-squares fit to sine and cosine waves without boxing the data like BLS.

In other words, RV data are noisy time-series, and real astronomical observations are often unevenly sampled as well. Lomb-Scargle is designed to find sinusoidal periodic signals in noisy, unevenly sampled data, which makes it the most appropriate for this use case.

```
In [28]: # Periodigram algorithm
from astropy.timeseries import LombScargle

ls = LombScargle(t_array, starRV_obs, RV_err)
frequency, power = ls.autopower()
period = frequency**(-1)
significance = ls.false_alarm_probability(power.max())
print(f"False Alarm Probability: {significance:.20%}")
```

False Alarm Probability: 0.00000000000000009843%

This false alarm probability tells me that it is almost guaranteed that the signal recovered by the algorithm are caused by a real planet and not just random noise

I've chosen to only plot Periods with $2 < P < 76$ days because we are only sampling once a day (so minimum two days to see a second transit) and for a window of 150 days (we can only see two transits of a planet if its period is less than half our observation window)

```
In [29]: # Plot the Periodigram
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

# Label known planet periods
ax.axvline(P_K2b.value, color="red", lw = 1.25, ls="--", label="K2-3b")
ax.axvline(P_K2c.value, color="blue", lw = 1.25, ls="--", label="K2-3c")
ax.axvline(P_K2d.value, color="green", lw = 1.25, ls="--", label="K2-3d")

ax.plot(period, power, color="black", lw=2)

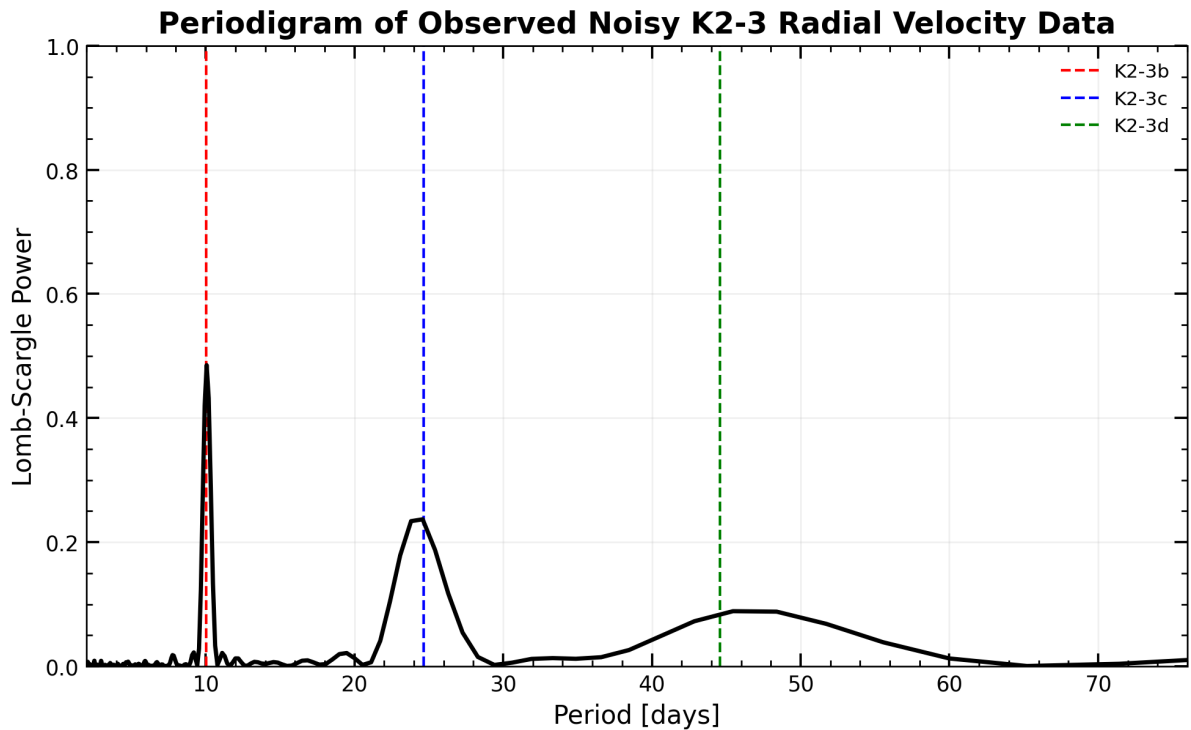
ax.set_xlabel("Period [days]", fontsize=12)
ax.set_ylabel("Lomb-Scargle Power", fontsize=12)
ax.set_title("Periodigram of Observed Noisy K2-3 Radial Velocity Data")

# axes limits chose from observational constraints
ax.set_xlim(2, 76)
ax.set_ylim(0,1)

# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, w
ax.tick_params(axis="both", which="minor", direction="in", length=3, w
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)
ax.legend(frameon=False, fontsize=9, loc="best")

fig.tight_layout()
plt.show()
```



Yes we detect the ~10 days, ~25 days and ~44 days periods of our three planets. Although it seems that the added noise makes it difficult to detect the exact value of the K2-3d period; the value doesn't match the local maximum. If we were to determine its period from the periodogram we would probably claim a value a couple days too long.

It is good to note that we have more repeating transits from the shorter period planets, meaning that the peak is narrower (higher SNR), whereas planets with long periods only get a few transits observed which makes the peaks broader (lower SNR).

g) Use any software or code of your choice to adjust a radial velocity curve to the simulated observations that you built in e). You can assume fixed values for the period of each three planets from the transit data. Estimate the error bars on your adjusted parameters with a method of your choice. The Systemic 2.0 software is able to do this with a Markov Chain Monte Carlo algorithm. If you write your own curve fitting routine, a simpler way to estimate the error bars is to repeat e) many times using different random values, and measure the scatter in the measured parameters across all realizations (i.e., a regular Monte Carlo method).

Thoughts: We are fitting a curve to the star RV curve which comes from the semi-

amplitudes of the three planets -> we are adjusting **K_K2b, K_K2c, K_K2d** values to change our fit. Makes sense to me since we get good periods and transit times from transit observations, whereas RV measurements give us orbit semi-amplitudes (from which we can then get the planet masses).

So the model fit will be a function such as **RV(time, K_b, K_c, K_d)**. We do MC to get uncertainties on these values of **K**.

I define the values we will keep fixed

```
In [30]: # Fixed values (will redefine some just to be explicit)
w_fit = 90                                # deg
e_fit = 0.0
v_sys_fit = 0.0 * (u.m / u.s)
t_fit = t_array.value                     # days
rv_fit = starRV_obs.value                 # m/s
rv_err_fit = RV_err.value                 # m/s

#P_array is already defined, given
#t_0_array is already defined, given along with Periods (I don't think
```

I define a wrapper function to my star RV function that only has my varying parameters as variables. This is the function scipy will use

```
In [31]: # Wrapper with parameters we change, the model scipy.curve_fit will use
def rv_model_fit(t, Kb, Kc, Kd):
    K_array_fit = np.array([Kb, Kc, Kd]) * (u.m / u.s)

    return TotalStarRV( K_array_fit,
                        w_fit,
                        v_sys_fit,
                        e_fit,
                        P_array, # will iterate over the three planets
                        t * u.day,
                        t_0_array
                        ).value
```

I define my initial guesses to be the semi-amplitudes we computed from the ideal model earlier, they are close (within an order of magnitude) to the values that should fit the noisy data best, ensuring convergence and that the value we converge to is the correct one.

```
In [32]: # initial guesses
initial_guess = [K_K2b.value, K_K2c.value, K_K2d.value]

popt, pcov = curve_fit( rv_model_fit,          # function to fit
                        t_fit,                  # xdata
                        rv_fit,                 # ydata
```

```

        sigma=rv_err_fit,      # y-uncertainties
        p0=initial_guess      # pass the initial guess
    )

# fit uncertainty, will not use. Will do MC to get uncertainties instead
fit_err = np.sqrt(np.diag(pcov))

print(f"Best fit values of the semi-amplitudes (K) from scipy.curve_fit")
print("-" * 64)
print(f"K2-3b: {popt[0]} m/s\nK2-3c: {popt[1]} m/s\nK2-3d: {popt[2]} m/s")
print(f"\nTheoretical values from model:")
print("-" * 64)
print(f"K2-3b: {K_K2b.value} m/s\nK2-3c: {K_K2c.value} m/s\nK2-3d: {K_K2d.value} m/s")

```

Best fit values of the semi-amplitudes (K) from scipy.curve_fit:

```

-----
K2-3b: 2.3217795598258486 m/s
K2-3c: 1.7006723843849492 m/s
K2-3d: 1.1319802433661805 m/s

```

Theoretical values from model:

```

-----
K2-3b: 2.270656819521336 m/s
K2-3c: 1.374393317381291 m/s
K2-3d: 1.005630202371288 m/s

```

To estimate the uncertainties of the fit, I do the same fit 1000 times (statistically significant sample) with slightly different random noise each trial (Monte Carlo method).

N.B. the given the seed only ensures that each time I run the notebook, the randomized values are in the same order, not that each randomization gives the same number (i.e. it will randomize to 12345, 23413, 34543, etc. each time we run the notebook, not 12345, 12345, 12345, etc.)

```

In [33]: # Monte Carlo setup
N_MC = 1000

# Gaussian parameters
stdev = 1.0 #m/s
mean = 0.0 #will center on RV value when added on top of data
sampling = np.arange(0,151,1,dtype=int) # 1 measurement per day for 151 days

MC_K_values = np.zeros((N_MC,3)) # 1000 rows, 3 columns

for i in tqdm(range(N_MC)):
    # Gaussian noise generates from the same rng seed that was done in the fit
    noise = rng.normal(
        loc=mean,
        scale=stdev,
    )

```

```

        size= len(sampling)
        ) * (u.m / u.s)

# Data with noise added
MC_RV = starRVtot + noise

# Curve fit
MC_popt, MC_pcov = curve_fit(
                        rv_model_fit,          # function to fit
                        t_fit,                 # xdata
                        MC_RV,                 # ydata
                        sigma=rv_err_fit,      # y-uncertainties
                        p0=initial_guess       # pass the initial guess
                    )

MC_K_values[i] = MC_popt

```

```
100%|██████████| 1000/1000 [00:11<00:00, 87.40it/s]
```

I take the mean and standard deviation of the resulting best fit parameters from the 1000 trials as my uncertainties

```

In [34]: # Measuring scatter in parameters
MC_mean = np.mean(MC_K_values, axis=0)
MC_std = np.std(MC_K_values, axis=0)

print(f"Monte Carlo Mean Results for the semi-amplitudes of the planet")
print("-" * 64)
print(f"K2-3b: {MC_mean[0]} m/s\nK2-3c: {MC_mean[1]} m/s\nK2-3d: {MC_m")
print(f"\nMonte Carlo Standard Deviation Results:")
print("-" * 64)
print(f"K2-3b: {MC_std[0]} m/s\nK2-3c: {MC_std[1]} m/s\nK2-3d: {MC_std

```

Monte Carlo Mean Results for the semi-amplitudes of the planets:

```

-----
K2-3b: 2.2725070170236714 m/s
K2-3c: 1.3749395044621902 m/s
K2-3d: 1.0114473807789865 m/s

```

Monte Carlo Standard Deviation Results:

```

-----
K2-3b: 0.1119335998997701 m/s
K2-3c: 0.11175133384289905 m/s
K2-3d: 0.11398783003306566 m/s

```

This is the curve given by the best fit parameters

```

In [35]: # compute best fit curve
MC_best_fit = TotalStarRV(popt * u.m / u.s, w_fit, v_sys_fit, e_fit, P

```

To account for uncertainty, I plot the curves given by the best fit values of $K \pm 1\sigma$ (a bit like bollinger bands)

```
In [36]: # compute best fit +/- stdev on the K parameters
MC_best_plus_stdev = TotalStarRV((popt + MC_std) * u.m / u.s, w_fit, v
MC_best_minus_stdev = TotalStarRV((popt - MC_std) * u.m / u.s, w_fit,
```

I plot the fit on top of the noisy data

```
In [37]: # Plot this fit to the noisy data generated in part e)
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

# Best fit to noisy data
ax.plot(t_array, MC_best_fit, color="red", lw=1, label="Best Fit", zor

# Uncertainties of best fit
ax.fill_between(
    t_array.value,
    MC_best_minus_stdev.value,
    MC_best_plus_stdev.value,
    color = "red",
    alpha = 0.25,
    label = r"$\pm$ 1 stdev uncertainty"
)

# Model line
#ax.plot(t_array, starRVtot, color="blue", lw=1, label="Model RV", zor

# Noisy data with Error bars
ax.errorbar(
    t_array.value,
    starRV_obs.value,
    yerr=RV_err.value,
    fmt="o",
    ms=3.0,
    color='black',
    alpha=.8,
    capsize=2,
    label="Observed Noisy RV",
    #lw = 1,
    #zorder = -10
)

ax.axhline(0, color='gray', lw=1, ls="--", alpha=0.3)

ax.set_xlabel("Time starting from K2-3c central transit [days]", fontst
ax.set_ylabel("Radial velocity [m/s]", fontsize=12)
ax.set_title("K2-3 Radial Velocity Curve with Best Fit", fontsize=14,

ax.set_xlim(t_array[0].value, t_array[-1].value)

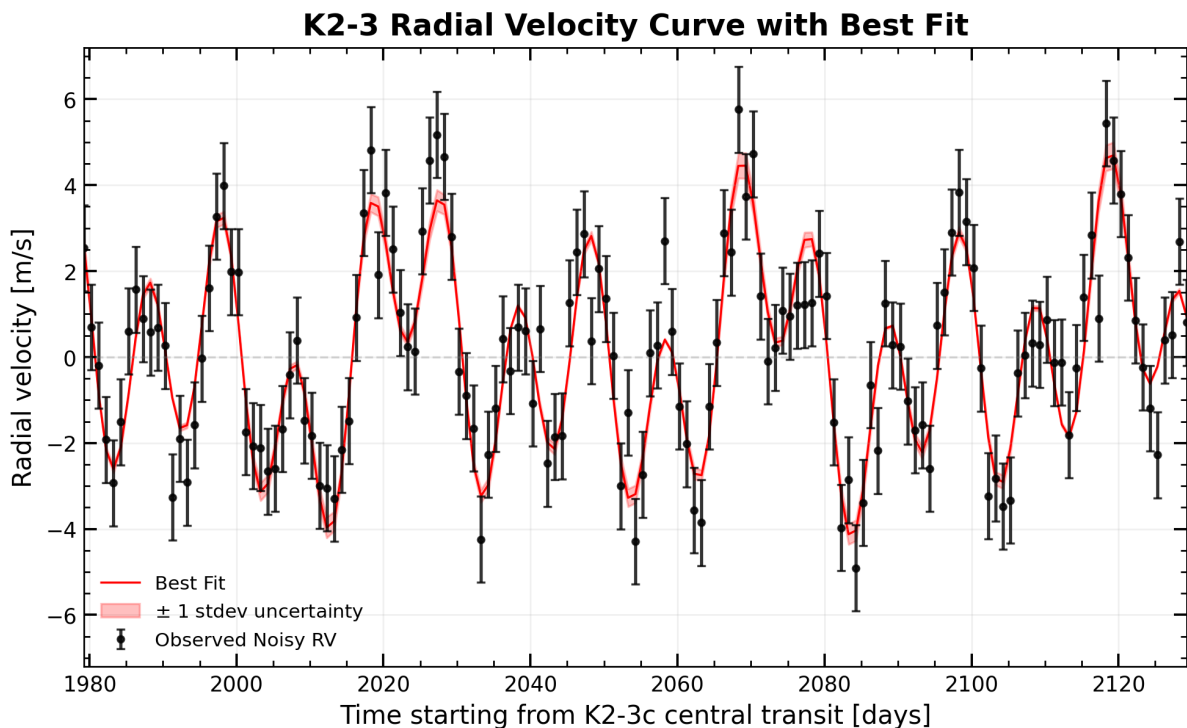
# make y-limits symmetric around zero
rv_absmax = max(abs(starRV_obs.value.min()), abs(starRV_obs.value.max(
ax.set_ylim(-1.25 * rv_absmax, 1.25 * rv_absmax)
```

```
# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, w
ax.tick_params(axis="both", which="minor", direction="in", length=3, w
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)

ax.legend(frameon=False, fontsize=9, loc="best")

fig.tight_layout()
plt.show()
```



h) Calculate the residuals of your best-fitting radial velocity curve (i.e., data - model) and build a periodogram for these residuals. Can you see any significant periodic signal in your residuals ?

The residuals are given by the difference between my noisy data and the best fit I just computed

```
In [38]: # Compute residuals
residuals = starRV_obs - MC_best_fit
```

uncertainties in the residuals are (± 1 m/s) from the observed data and (± 0.1 m/s) from the best fit so I'll just use the noisy data error bars for simplicity since

adding them in quadrature would make the fit errors even smaller, thus the dominant uncertainty is the one stemming from the ± 1 m/s precision of our instrument.

```
In [39]: # plot residuals
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

# Residuals with Error bars that come from the data
ax.errorbar(
    t_array.value,
    residuals.value,
    yerr=RV_err.value,
    fmt=".",
    ms=3.0,
    color='black',
    alpha=.8,
    capsize=2,
    label="Residuals",
    #lw = 1,
    #zorder = -10
)

ax.axhline(0, color='gray', lw=1, ls="--", alpha=0.3)

ax.set_xlabel("Time starting from K2-3c central transit [days]", fontsize=12)
ax.set_ylabel("Radial velocity [m/s]", fontsize=12)
ax.set_title("K2-3 Radial Velocity Curve Residuals", fontsize=14, fontweight='bold')

ax.set_xlim(t_array[0].value, t_array[-1].value)

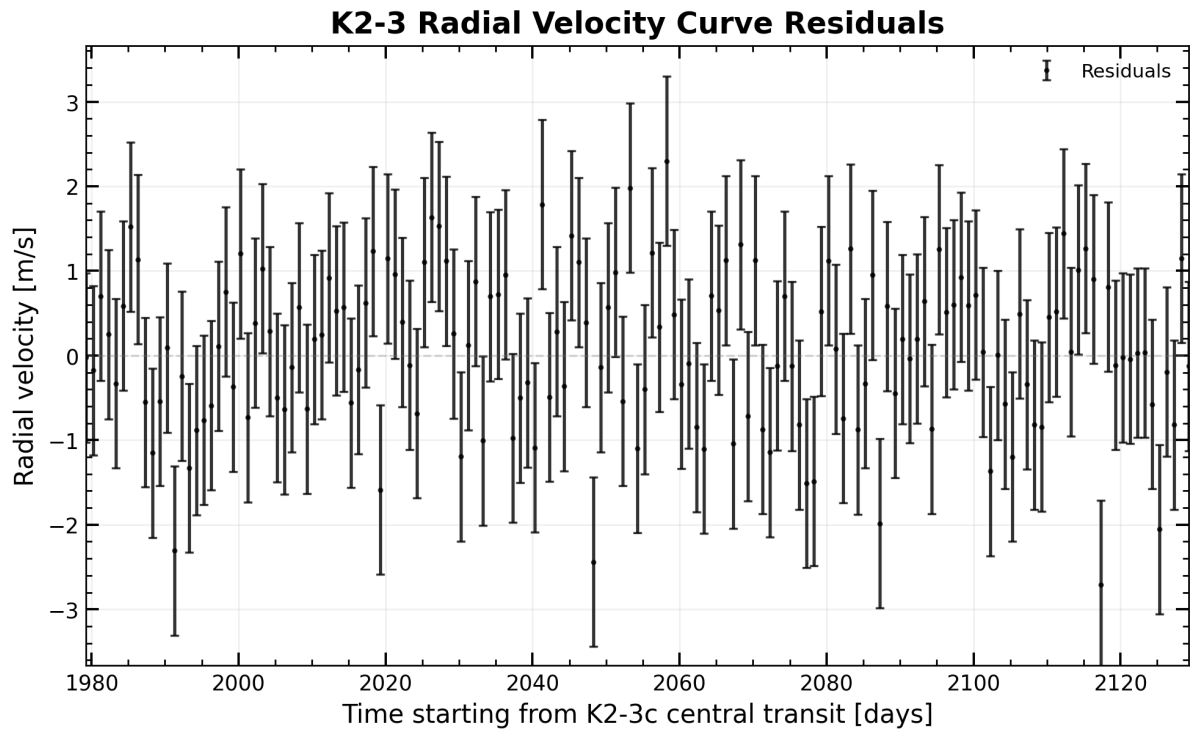
# make y-limits symmetric around zero
rv_absmax = max(abs(residuals.value.min()), abs(residuals.value.max()))
ax.set_ylim(-1.35 * rv_absmax, 1.35 * rv_absmax)

# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, width=1)
ax.tick_params(axis="both", which="minor", direction="in", length=3, width=1)
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)

ax.legend(frameon=False, fontsize=9, loc="best")

fig.tight_layout()
plt.show()
```



The residuals may look alarming at first since they have amplitudes that are within the same order of magnitude as our measurements, however I show below that the distribution of these points is indeed random and that there is no signal to recover in these residuals.

```
In [40]: # Lomb-Scargle Periodogram
ls = LombScargle(t_array, residuals, RV_err)
frequency, power = ls.autopower()
period = frequency**(-1)
significance = ls.false_alarm_probability(power.max())
print(f"False Alarm Probability: {significance:.20%}")
```

False Alarm Probability: 98.78527020981859152471%

This false alarm probability means that there is a ~98% chance the peaks in the periodogram could be due to random noise in the data. Which is what we want, if it were otherwise it would suggest that we missed a physical signal in our data recovery.

```
In [41]: # Plot the Periodogram
fig, ax = plt.subplots(figsize=(8, 5), dpi=250)

# Label known planet periods
ax.axvline(P_K2b.value, color="red", lw = 1.25, ls="--", label="K2-3b")
ax.axvline(P_K2c.value, color="blue", lw = 1.25, ls="--", label="K2-3c")
ax.axvline(P_K2d.value, color="green", lw = 1.25, ls="--", label="K2-3d")

ax.plot(period, power, color="black", lw=2)
```

```

ax.set_xlabel("Period [days]", fontsize=12)
ax.set_ylabel("Lomb-Scargle Power", fontsize=12)
ax.set_title("Periodogram of K2-3 Residuals", fontsize=14, fontweight=

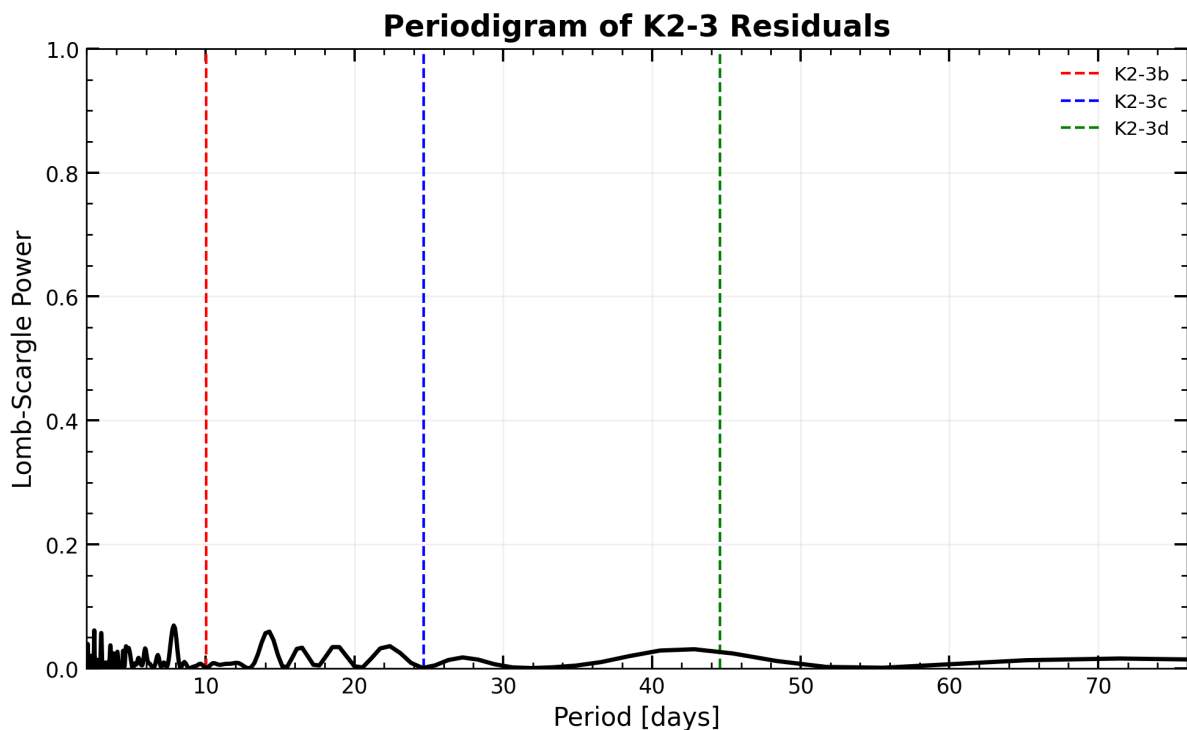
ax.set_xlim(2, 76)
ax.set_ylim(0,1)

# ticks on all four sides
ax.tick_params(axis="both", which="major", direction="in", length=6, w
ax.tick_params(axis="both", which="minor", direction="in", length=3, w
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())

ax.grid(True, which="major", alpha=0.18, lw=0.8)
ax.legend(frameon=False, fontsize=9, loc="best")

fig.tight_layout()
plt.show()

```



In addition to the extremely high false alarm probability, the max LS power in the residuals is < 0.1 , thus I claim that there are no significant periodic signals in my residuals, meaning the fit to the data is good.

i) What conclusions would you draw from this exercise about the feasibility of detecting this planetary system with a 1 m/s precision? How feasible would it be to detect these planets with the radial velocity method with a degraded precision of 2 m/s? How feasible would it be to detect these planets with a 1 m/s measurement precision if

their arguments of pericentre were all fixed at 180 degrees and their eccentricities were fixed at $e = 0.8$?

1. I draw the conclusion that detecting this planetary system with a 1 m/s precision is indeed feasible but non-trivial. Despite adding 1 m/s noise we were still able to recover semi-amplitudes for the planets as well as their periods. However, the RV signal from each planet is roughly 1-2 m/s which is on the same order of magnitude as the noise, which makes detection harder, however with 150 days of observation and known transit periods/epochs like we have here, we are able to confidently constrain the planets' semi-amplitudes and masses.

2. I would argue that if the precision degraded to 2 m/s it would be much less feasible to detect this planetary system. This is because the noise would now be larger than most of the planets' semi-amplitudes, so that the individual planet signals would be strongly buried in the noise. The planet with the strongest signal may still be recoverable, but confidently measuring all three planet's masses would be challenging.

3. If we still had a 1 m/s precision but $\omega = 180^\circ$ and $e = 0.8$ this would make things both easier and more difficult at the same time. Since the semi-amplitude scales as $K \propto \frac{1}{\sqrt{1-e^2}}$, the higher eccentricity would make the values of K larger and thus easier to detect, however said signal would be concentrated at the part of the orbit that is close to the star. This means that depending on when we observe the system, the feasibility may change drastically. Moreover, an argument of pericenter of 180° will change where the planet's pericenter is relative to our line of sight, and thus affect the phase and shape of the RV curve. This means the star's RV won't be zero when it transits anymore.